APB Protocol Verification using UVM

Command line for Run code

[b23n_sarif_727@prac3 apb_protocol]$ ls

agent.sv apb_pkg.sv driver.sv monitor.sv novas.rc sequencer.sv simv.cst test.sv verdiLog

apb_dump.fsdb apb_slave.sv environment.sv novas.conf scoreboard.sv sequence.sv simv.daidir ucli.key

apb_if.sv csrc inter.fsdb novas_dump.log sequence_item.sv simv tb_top.sv vc_hdrs.h

Design Code

```
//

module apb_slave #(parameter integer WAIT_CYCLES = 2)(

        input logic       pclk,

      input logic       rst_n,

      input logic [31:0] paddr,

      input logic       psel,

      input logic       penable,

      input logic       pwrite,

      input logic [31:0] pwdata,

        output logic [31:0]prdata,

      output logic      pready,

      output logic      pslverr

);


logic [31:0] mem [0:255];
```

```
typedef enum logic [1:0] { SETUP = 2'b00, W_ENABLE = 2'b01, R_ENABLE = 2'b10 } state_t;

state_t apb_st;

integer wait_count;


always_ff @(posedge pclk or negedge rst_n) begin
        if (!rst_n) begin
                apb_st     <= SETUP;
                prdata     <= 32'd0;
            pready     <= 1'b1;
            pslverr    <= 1'b0;
            wait_count <= 0;

                for (int i = 0; i < 256; i++)
                        mem[i] <= i;
        end

        else begin
                case (apb_st)
                        SETUP:
                        begin
                                pready  <= 1'b0;
                                pslverr <= 1'b0;
                                prdata  <= 32'd0;

                                if (psel && !penable) begin
```

```
                if (paddr > 32'd255) begin
                        pslverr <= 1'b1;
                        pready  <= 1'b1;
                        apb_st <= SETUP;
                end
                else begin
                        wait_count <= WAIT_CYCLES;
                        if (pwrite)
                                apb_st <= W_ENABLE;
                        else
                                apb_st <= R_ENABLE;
                end
        end
end
W_ENABLE:
begin
        if (psel && penable && pwrite) begin
                if (wait_count > 0) begin
                        pready     <= 1'b0;
                        wait_count <= wait_count - 1;
                end
                else begin
                        pready <= 1'b1;
                        if (paddr <= 32'd255)
                                mem[paddr] <= pwdata;
```

```
                    apb_st <= SETUP;
                end
            end
        end
        R_ENABLE:
        begin
            if (psel && penable && !pwrite) begin
                if (wait_count > 0) begin
                    pready     <= 1'b0;
                    wait_count <= wait_count - 1;
                end
                else begin
                    pready <= 1'b1;
                    if (paddr <= 32'd255)
                        prdata <= mem[paddr];
                    apb_st <= SETUP;
                end
            end
        end
        default:
        begin
            apb_st     <= SETUP;
            pready     <= 1'b1;
            pslverr    <= 1'b0;
            prdata     <= 32'd0;
```

```
                    wait_count <= 0;
                end
            endcase
        end
end
endmodule

//
class apb_seq_item extends uvm_sequence_item;
        rand bit [31:0] paddr;
        rand bit        pwrite;
        rand bit [31:0] pwdata;
        bit [31:0] prdata;
      bit       pready;
      bit       pslverr;

        `uvm_object_utils_begin(apb_seq_item)
        `uvm_field_int(paddr,   UVM_ALL_ON)
      `uvm_field_int(pwrite,  UVM_ALL_ON)
      `uvm_field_int(pwdata,  UVM_ALL_ON)
      `uvm_field_int(prdata,  UVM_ALL_ON)
      `uvm_field_int(pready,  UVM_ALL_ON)
      `uvm_field_int(pslverr, UVM_ALL_ON)
        `uvm_object_utils_end
```

```
	function new(string name = "apb_seq_item");
		super.new(name);
	endfunction

endclass

//
class apb_base_seq extends uvm_sequence #(apb_seq_item);
	apb_seq_item req;
	`uvm_object_utils(apb_base_seq)

	function new(string name = "apb_base_seq");
		super.new(name);
	endfunction

	task body();
		repeat(10) begin
			`uvm_do_with(req, { pwrite == 1; paddr inside {[0:255]}; })
		end

		repeat(10) begin
			`uvm_do_with(req, { pwrite == 0; paddr inside {[0:255]}; })
		end

		`uvm_do_with(req, { pwrite == 0; paddr > 255; })
```

```
        endtask
endclass

//
class apb_seqcr extends uvm_sequencer #(apb_seq_item);
        `uvm_component_utils(apb_seqcr)

        function new(string name = "apb_seqcr", uvm_component parent = null);
                super.new(name, parent);
        endfunction

endclass

//
class apb_driver extends uvm_driver #(apb_seq_item);
        virtual apb_if vif;
        `uvm_component_utils(apb_driver)

        function new(string name = "apb_driver", uvm_component parent = null);
                super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                if (!uvm_config_db#(virtual apb_if)::get(this, "", "vif", vif))
```

```
            `uvm_fatal(get_type_name(), "Virtual interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        forever begin
            seq_item_port.get_next_item(req);
            drive_to_dut(req);
            seq_item_port.item_done();
        end
    endtask

    task drive_to_dut(apb_seq_item req);
        @(posedge vif.pclk);
        vif.psel    <= 1'b1;
        vif.penable <= 1'b0;
        vif.paddr   <= req.paddr;
        vif.pwrite  <= req.pwrite;
        vif.pwdata  <= req.pwdata;
        `uvm_info(get_type_name(), $sformatf("SETUP: addr=%0d write=%0d data=%0h",req.paddr,req.pwrite,req.pwdata),UVM_LOW)

        @(posedge vif.pclk);
        vif.penable <= 1'b1;

    while (!vif.pready) begin
        @(posedge vif.pclk);
```

```
                `uvm_info(get_type_name(), "Waiting for PREADY...",
UVM_HIGH)
        end


      req.prdata  = vif.prdata;
      req.pready  = vif.pready;
      req.pslverr = vif.pslverr;
        `uvm_info(get_type_name(),$sformatf("ACCESS DONE: addr=%0d
pwrite=%0d prdata=%0h
PSLVERR=%0d",req.paddr,req.pwrite,req.prdata,req.pslverr),UVM_LOW)


        @(posedge vif.pclk);
      vif.psel    <= 1'b0;
      vif.penable <= 1'b0;
      vif.paddr   <= 32'd0;
      vif.pwdata  <= 32'd0;
endtask
endclass

//
class apb_monitor extends uvm_monitor;
        virtual apb_if vif;
        uvm_analysis_port #(apb_seq_item) mon_ap;
        `uvm_component_utils(apb_monitor)

        function new(string name = "apb_monitor", uvm_component parent = null);
```

```
        super.new(name, parent);
        mon_ap = new("mon_ap", this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual apb_if)::get(this, "", "vif", vif))
            `uvm_fatal(get_type_name(),"Virtual interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        apb_seq_item mon_item;
        forever begin
            @(posedge vif.pclk);
            if (vif.psel && vif.penable && vif.pready) begin
                mon_item = apb_seq_item::type_id::create("mon_item");

                mon_item.paddr   = vif.paddr;
                mon_item.pwrite  = vif.pwrite;
                mon_item.pwdata  = vif.pwdata;
                mon_item.prdata  = vif.prdata;
                mon_item.pready  = vif.pready;
                mon_item.pslverr = vif.pslverr;
                `uvm_info(get_type_name(),$sformatf("MONITOR:
addr=%0d write=%0d pwdata=%0h prdata=%0h
```

```
PSLVERR=%0d",mon_item.paddr,mon_item.pwrite,mon_item.pwdata,mon_item.
prdata,mon_item.pslverr),UVM_HIGH)
                        mon_ap.write(mon_item);
                    end
                end
            endtask
endclass

//
class apb_scoreboard extends uvm_scoreboard;
    uvm_analysis_imp #(apb_seq_item, apb_scoreboard) sb_imp;
    bit [31:0] ref_mem [0:255];
    `uvm_component_utils(apb_scoreboard)

    function new(string name = "apb_scoreboard", uvm_component parent =
null);
        super.new(name, parent);
        sb_imp = new("sb_imp", this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);

        for (int i = 0; i < 256; i++)
            ref_mem[i] = i;
    endfunction
```

```
    function void write(apb_seq_item req);
        if (req.paddr > 255) begin
            if (req.pslverr == 1'b1) begin
                `uvm_info(get_type_name(),$sformatf("INVALID
ADDRESS PASS: addr=%0d
PSLVERR=%0d",req.paddr,req.pslverr),UVM_LOW)
            end
            else begin
                `uvm_error(get_type_name(),$sformatf("INVALID
ADDRESS FAIL: addr=%0d PSLVERR=%0d",req.paddr,req.pslverr))
            end
            return;
        end

        if (req.pwrite) begin
            ref_mem[req.paddr] = req.pwdata;
            `uvm_info(get_type_name(),$sformatf("WRITE PASS:
addr=%0d data=%0h",req.paddr,req.pwdata),UVM_LOW)
        end
        else begin
            if (req.prdata == ref_mem[req.paddr]) begin
                `uvm_info(get_type_name(),$sformatf("READ PASS:
addr=%0d expected=%0h
actual=%0h",req.paddr,ref_mem[req.paddr],req.prdata),UVM_LOW)
            end
            else begin
```

```
                `uvm_error(get_type_name(),$sformatf("READ FAIL:
addr=%0d expected=%0h actual=%0h",req.paddr,ref_mem[req.paddr],req.prdata))
                    end
                end
        endfunction

endclass

//
class apb_agent extends uvm_agent;
        `uvm_component_utils(apb_agent)
        apb_driver drv;
        apb_seqcr  seqr;
        apb_monitor mon;

        function new(string name = "apb_agent", uvm_component parent = null);
                super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                mon = apb_monitor::type_id::create("mon", this);
                if (get_is_active() == UVM_ACTIVE) begin
                        drv = apb_driver::type_id::create("drv", this);
                        seqr = apb_seqcr::type_id::create("seqr", this);
                end
```

```
        endfunction

        function void connect_phase(uvm_phase phase);
                super.connect_phase(phase);
                if (get_is_active() == UVM_ACTIVE) begin
                        drv.seq_item_port.connect(seqr.seq_item_export);
                end
        endfunction

endclass

//
class apb_env extends uvm_env;
        `uvm_component_utils(apb_env)
        apb_agent      agt;
        apb_scoreboard sb;

        function new(string name = "apb_env", uvm_component parent = null);
                super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                agt = apb_agent::type_id::create("agt", this);
                sb = apb_scoreboard::type_id::create("sb", this);
```

```
        endfunction

        function void connect_phase(uvm_phase phase);
                super.connect_phase(phase);
                agt.mon.mon_ap.connect(sb.sb_imp);
        endfunction

endclass

//
class apb_base_test extends uvm_test;
        apb_env      env_o;
        apb_base_seq bseq;
        `uvm_component_utils(apb_base_test)

        function new(string name = "apb_base_test", uvm_component parent =
null);
                super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                env_o = apb_env::type_id::create("env_o", this);
        endfunction

        task run_phase(uvm_phase phase);
```

```
        phase.raise_objection(this);
        bseq = apb_base_seq::type_id::create("bseq");
        bseq.start(env_o.agt.seqr);
        #50;
        phase.drop_objection(this);

        `uvm_info(get_type_name(),"END OF TESTCASE",UVM_LOW)
    endtask

endclass

//
`include "uvm_macros.svh"
import uvm_pkg::*;
import apb_pkg::*;

module tb_top;

bit pclk;
always #5 pclk = ~pclk;

bit rst_n;

initial begin
    rst_n = 1'b0;
```

```
        #20;
        rst_n = 1'b1;
end

apb_if vif(.pclk (pclk),.rst_n (rst_n));

apb_slave #(.WAIT_CYCLES(2)) DUT (.pclk (vif.pclk),.rst_n (vif.rst_n),.paddr
(vif.paddr),.psel (vif.psel),.penable (vif.penable),.pwrite (vif.pwrite),.pwdata
(vif.pwdata),.prdata (vif.prdata),.pready (vif.pready),.pslverr (vif.pslverr));

initial begin
        uvm_config_db#(virtual apb_if)::set( null,"*","vif",vif);
end
initial begin
        run_test("apb_base_test");
end
initial begin
        $fsdbDumpfile("apb_dump.fsdb");
      $fsdbDumpvars(0, tb_top);
end

endmodule

//
interface apb_if(input logic pclk, rst_n);
        logic [31:0] paddr;
```

```
    logic        psel;
    logic        penable;
    logic        pwrite;
    logic [31:0] pwdata;

    logic [31:0] prdata;
    logic        pready;
    logic        pslverr;

endinterface
//
package apb_pkg;
     `include "uvm_macros.svh"
     import uvm_pkg::*;
     `include "sequence_item.sv"
    `include "sequence.sv"
    `include "sequencer.sv"
    `include "driver.sv"
    `include "monitor.sv"
    `include "scoreboard.sv"
    `include "agent.sv"
    `include "environment.sv"
    `include "test.sv"
endpackage
//
```

```
class apb_coverage extends uvm_subscriber #(apb_seq_item);
        `uvm_component_utils(apb_coverage)

         bit        cov_pwrite;
       bit [31:0] cov_paddr;
       bit [31:0] cov_pwdata;
       bit [31:0] cov_prdata;
       bit        cov_pready;
       bit        cov_pslverr;

         covergroup apb_cg;

       cp_operation:
          coverpoint cov_pwrite {
             bins READ  = {0};
             bins WRITE = {1};
          }

       cp_address:
          coverpoint cov_paddr {
             bins LOW_ADDR  = {[0:63]};
             bins MID_ADDR  = {[64:191]};
             bins HIGH_ADDR = {[192:255]};
             bins INVALID   = {[256:$]};
          }
```

```
cp_boundary_address:
    coverpoint cov_paddr {
        bins ADDR_0   = {0};
        bins ADDR_1   = {1};
        bins ADDR_254 = {254};
        bins ADDR_255 = {255};
        bins INVALID  = {[256:$]};
    }

cp_pwdata:
    coverpoint cov_pwdata {
        bins ZERO      = {32'h00000000};
        bins MAX       = {32'hFFFFFFFF};
        bins ALL_A     = {32'hAAAAAAAA};
        bins ALL_5     = {32'h55555555};
        bins LOW_VALUE = {[1:255]};
        bins OTHER     = default;
    }

cp_prdata:
    coverpoint cov_prdata {
        bins ZERO      = {32'h00000000};
        bins MAX       = {32'hFFFFFFFF};
        bins LOW_VALUE = {[1:255]};
```

```
        bins OTHER     = default;
    }

cp_pready:
    coverpoint cov_pready {
        bins READY    = {1};
        bins NOT_READY = {0};
    }

cp_pslverr:
    coverpoint cov_pslverr {
        bins NO_ERROR = {0};
        bins ERROR    = {1};
    }

cross_operation_address:
    cross cp_operation, cp_address;

cross_operation_boundary:
    cross cp_operation, cp_boundary_address;

cross_operation_data:
    cross cp_operation, cp_pwdata;

cross_operation_pready:
```

```
        cross cp_operation, cp_pready;

    cross_operation_error:
        cross cp_operation, cp_pslverr;

    cross_address_error:
        cross cp_address, cp_pslverr;

    cross_address_pready:
        cross cp_address, cp_pready;

    cross_operation_address_error:
        cross cp_operation, cp_address, cp_pslverr;

    cross_operation_address_pready:
        cross cp_operation, cp_address, cp_pready;

    cross_write_addr_data:
        cross cp_operation, cp_address, cp_pwdata;

endgroup

function new(string name = "apb_coverage", uvm_component parent = null);
          super.new(name, parent);
          apb_cg = new();
```

```
    endfunction
    function void write(apb_seq_item t);
            cov_pwrite  = t.pwrite;
          cov_paddr   = t.paddr;
          cov_pwdata  = t.pwdata;
          cov_prdata  = t.prdata;
          cov_pready  = t.pready;
          cov_pslverr = t.pslverr;

            apb_cg.sample();
    endfunction
endclass
```

Run-Command

[b23n_sarif_727@prac3 apb_protocol]$ vcs -sverilog apb_slave.sv apb_if.sv apb_pkg.sv tb_top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 apb_protocol]$ ./simv

[b23n_sarif_727@prac3 apb_protocol]$ ./simv -verdi

Simulation Outputs

UVM_INFO driver.sv(31) @ 5: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=86 write=1 data=3e08d426

UVM_INFO driver.sv(44) @ 15: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=86 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO scoreboard.sv(32) @ 25: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=86 data=3e08d426

UVM_INFO driver.sv(31) @ 35: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=240 write=1 data=c6d1d15e

UVM_INFO scoreboard.sv(32) @ 85: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=240 data=c6d1d15e

UVM_INFO driver.sv(44) @ 85: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=240 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 105: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=73 write=1 data=f1d20d11

UVM_INFO scoreboard.sv(32) @ 155: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=73 data=f1d20d11

UVM_INFO driver.sv(44) @ 155: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=73 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 175: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=191 write=1 data=8dc5a05a

UVM_INFO scoreboard.sv(32) @ 225: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=191 data=8dc5a05a

UVM_INFO driver.sv(44) @ 225: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=191 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 245: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=116 write=1 data=5e8b780d

UVM_INFO scoreboard.sv(32) @ 295: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=116 data=5e8b780d

UVM_INFO driver.sv(44) @ 295: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=116 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 315: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=13 write=1 data=5c213a1d

UVM_INFO scoreboard.sv(32) @ 365: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=13 data=5c213a1d

UVM_INFO driver.sv(44) @ 365: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=13 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 385: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=211 write=1 data=1225fc42

UVM_INFO scoreboard.sv(32) @ 435: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=211 data=1225fc42

UVM_INFO driver.sv(44) @ 435: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=211 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 455: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=83 write=1 data=e959b00e

UVM_INFO scoreboard.sv(32) @ 505: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=83 data=e959b00e

UVM_INFO driver.sv(44) @ 505: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=83 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 525: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=58 write=1 data=93e4ece1

UVM_INFO scoreboard.sv(32) @ 575: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=58 data=93e4ece1

UVM_INFO driver.sv(44) @ 575: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=58 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 595: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=71 write=1 data=abb0b331

UVM_INFO scoreboard.sv(32) @ 645: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=71 data=abb0b331

UVM_INFO driver.sv(44) @ 645: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=71 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 665: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=148 write=0 data=4afd08d5

UVM_INFO scoreboard.sv(36) @ 715: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=148 expected=94 actual=94

UVM_INFO driver.sv(44) @ 715: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=148 pwrite=0 prdata=94 PSLVERR=0

UVM_INFO driver.sv(31) @ 735: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=54 write=0 data=4a272a28

UVM_INFO scoreboard.sv(36) @ 785: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=54 expected=36 actual=36

UVM_INFO driver.sv(44) @ 785: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=54 pwrite=0 prdata=36 PSLVERR=0

UVM_INFO driver.sv(31) @ 805: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=61 write=0 data=3d481d16

UVM_INFO scoreboard.sv(36) @ 855: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=61 expected=3d actual=3d

UVM_INFO driver.sv(44) @ 855: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=61 pwrite=0 prdata=3d PSLVERR=0

UVM_INFO driver.sv(31) @ 875: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=10 write=0 data=29a690a2

UVM_INFO scoreboard.sv(36) @ 925: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=10 expected=a actual=a

UVM_INFO driver.sv(44) @ 925: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=10 pwrite=0 prdata=a PSLVERR=0

UVM_INFO driver.sv(31) @ 945: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=26 write=0 data=350e24b6

UVM_INFO scoreboard.sv(36) @ 995: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=26 expected=1a actual=1a

UVM_INFO driver.sv(44) @ 995: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=26 pwrite=0 prdata=1a PSLVERR=0

UVM_INFO driver.sv(31) @ 1015: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=90 write=0 data=9d117889

UVM_INFO scoreboard.sv(36) @ 1065: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=90 expected=5a actual=5a

UVM_INFO driver.sv(44) @ 1065: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=90 pwrite=0 prdata=5a PSLVERR=0

UVM_INFO driver.sv(31) @ 1085: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=35 write=0 data=4e2ce6a9

UVM_INFO scoreboard.sv(36) @ 1135: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=35 expected=23 actual=23

UVM_INFO driver.sv(44) @ 1135: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=35 pwrite=0 prdata=23 PSLVERR=0

UVM_INFO driver.sv(31) @ 1155: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=166 write=0 data=da704690

UVM_INFO scoreboard.sv(36) @ 1205: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=166 expected=a6 actual=a6

UVM_INFO driver.sv(44) @ 1205: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=166 pwrite=0 prdata=a6 PSLVERR=0

UVM_INFO driver.sv(31) @ 1225: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=37 write=0 data=fb4bb2e7

UVM_INFO scoreboard.sv(36) @ 1275: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=37 expected=25 actual=25

UVM_INFO driver.sv(44) @ 1275: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=37 pwrite=0 prdata=25 PSLVERR=0

UVM_INFO driver.sv(31) @ 1295: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=115 write=0 data=6fd565b6

UVM_INFO scoreboard.sv(36) @ 1345: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=115 expected=73 actual=73

UVM_INFO driver.sv(44) @ 1345: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=115 pwrite=0 prdata=73 PSLVERR=0

UVM_INFO driver.sv(31) @ 1365: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=1060706736 write=0 data=bea2638

UVM_INFO scoreboard.sv(22) @ 1385: uvm_test_top.env_o.sb [apb_scoreboard] INVALID ADDRESS PASS: addr=1060706736 PSLVERR=1

UVM_INFO driver.sv(44) @ 1385: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=1060706736 pwrite=0 prdata=0 PSLVERR=1

UVM_INFO test.sv(23) @ 1445: uvm_test_top [apb_base_test] END OF TESTCASE

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 1445: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 66

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[TEST_DONE] 1

[apb_base_test] 1

[apb_driver] 42

[apb_scoreboard] 21

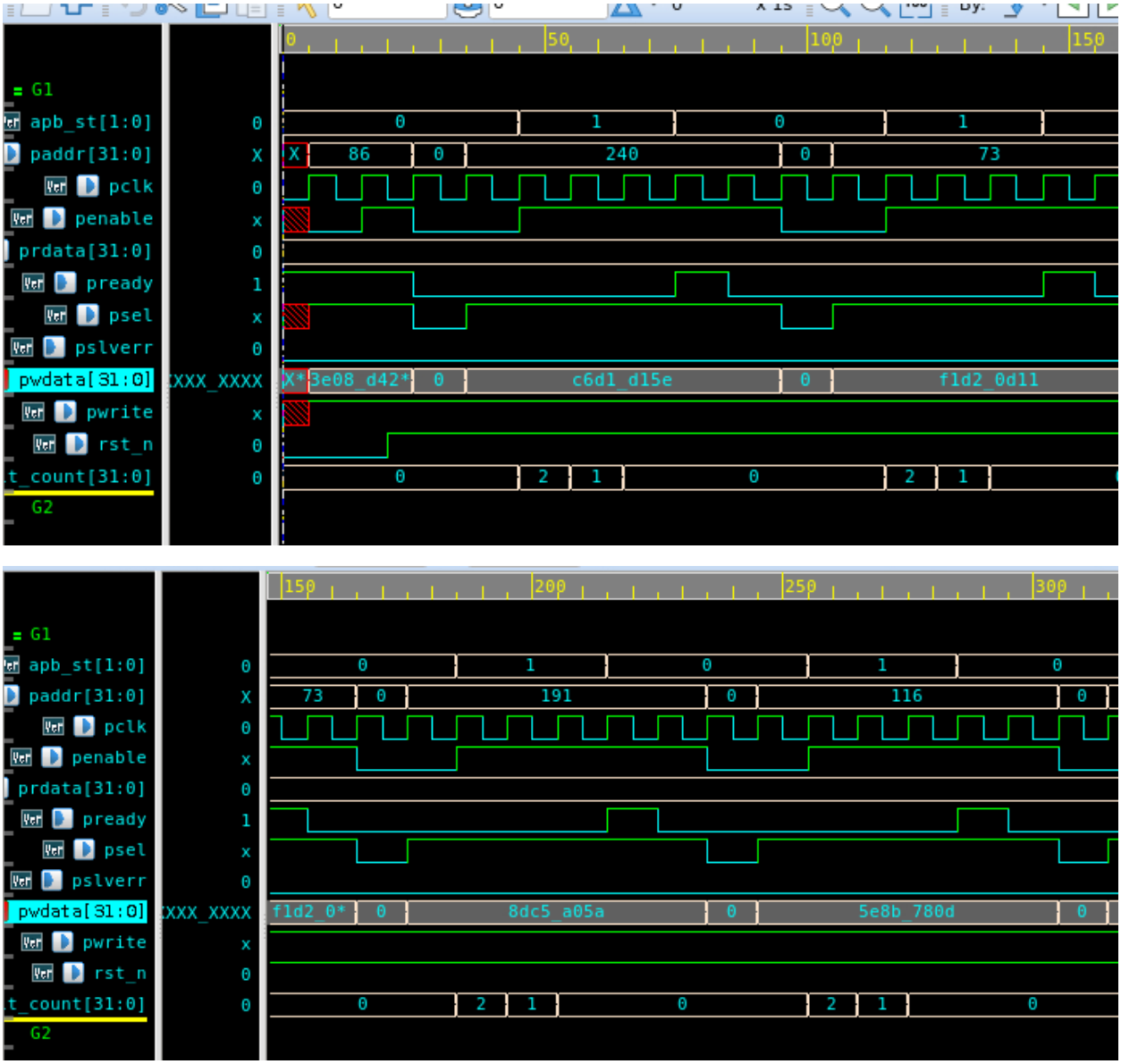
$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

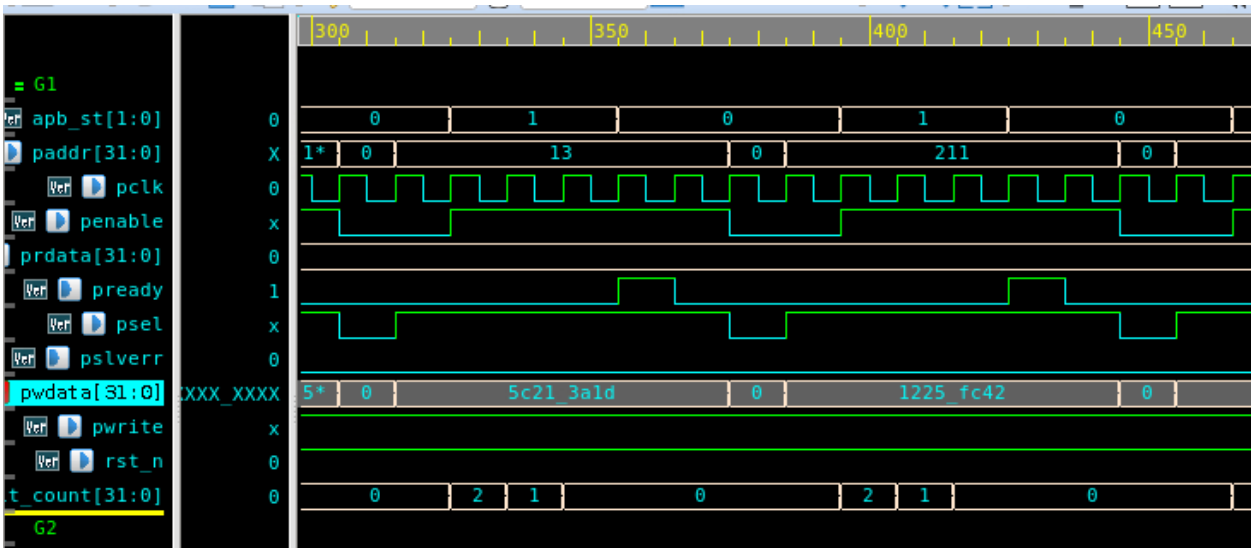
$finish at simulation time 1445

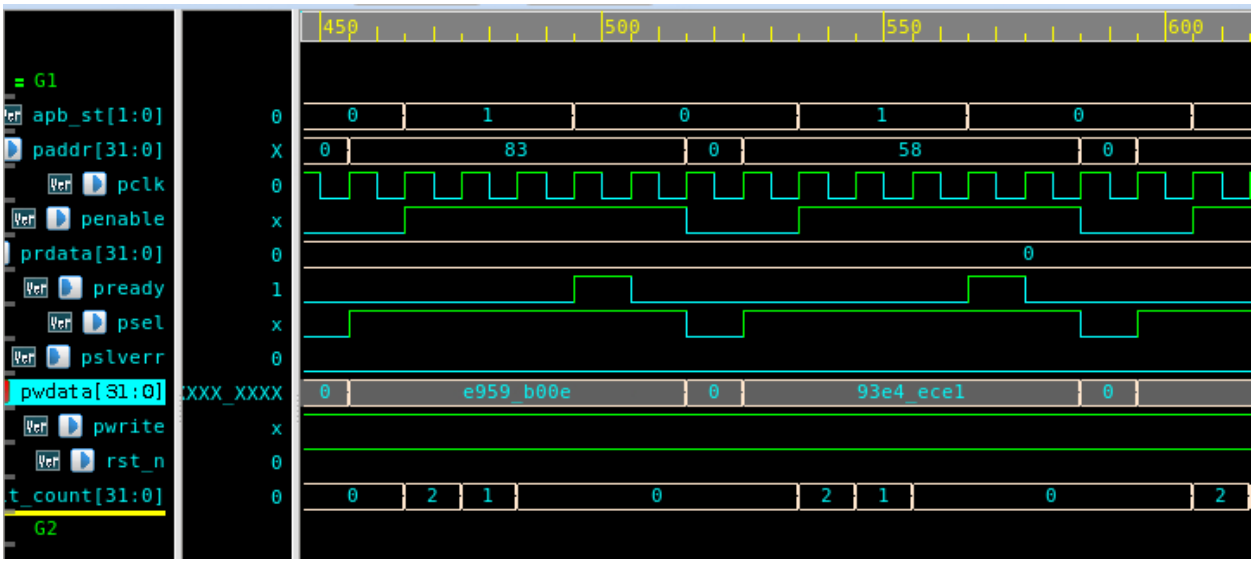
V C S S i m u l a t i o n R e p o r t

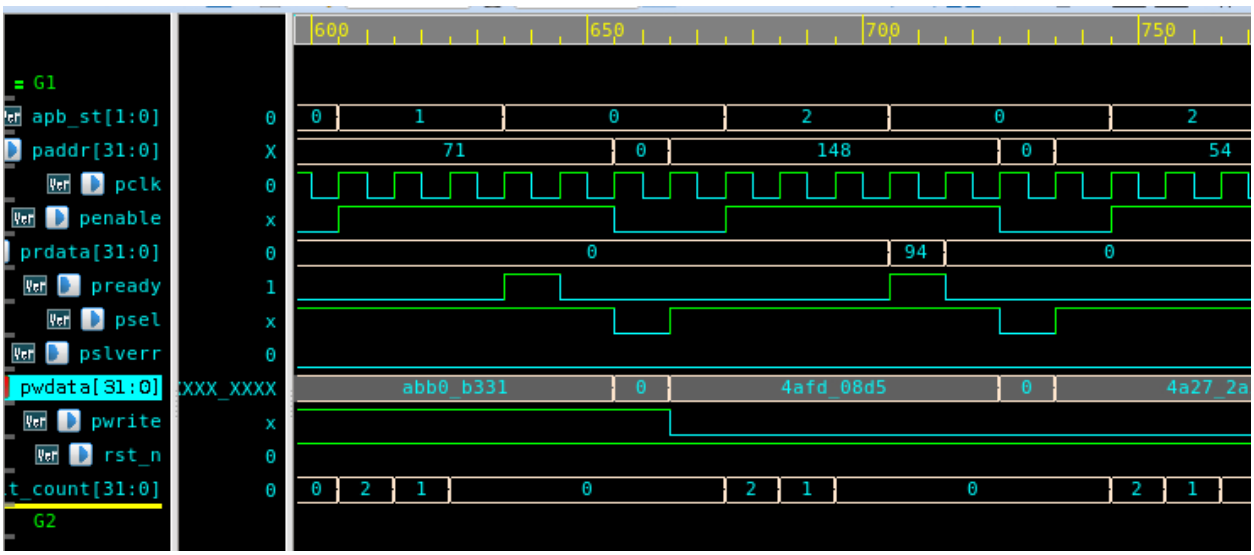
Time: 1445

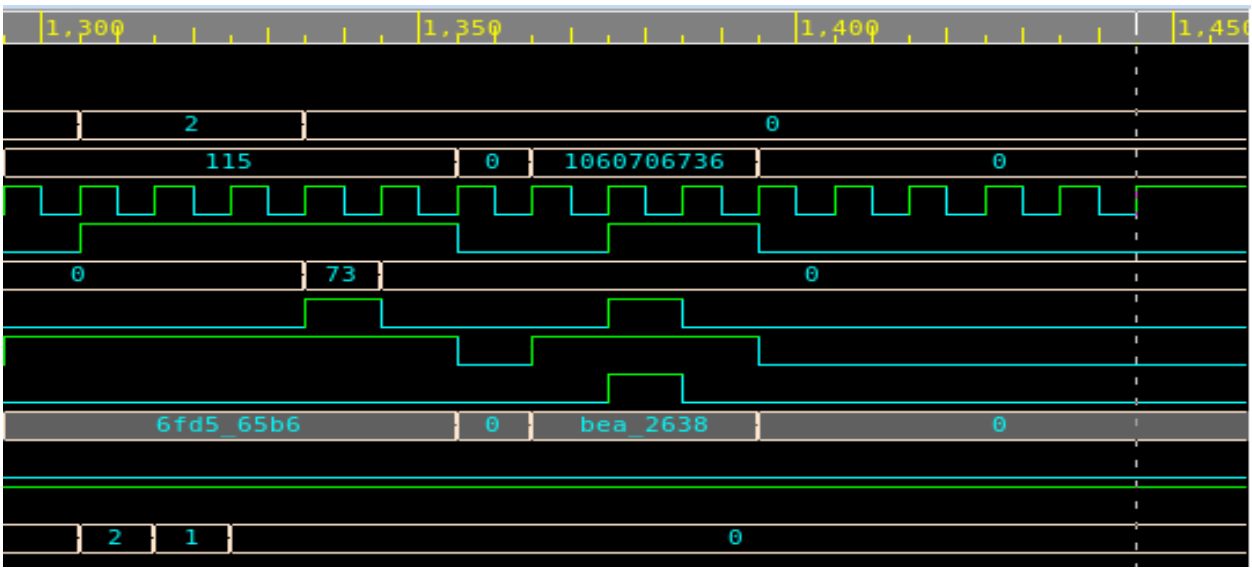
Simulation waveform


= G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
0
50
100
150
0 1 0 1
X 86 0 240 0 73
X* 3e08_d42* 0 c6d1_d15e 0 f1d2_0d11
0 2 1 0 2 1
150
200
250
300
0 1 0 1 0
73 0 191 0 116 0
f1d2_0* 0 8dc5_a05a 0 5e8b_780d 0
0 2 1 0 2 1 0


300
350
400
450
= G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
0
X
0
x
0
1
x
0
x
0
XXX_XXXX
x
0
0
0 1 0 1 0
1* 0 13 0 211 0
5* 0 5c21_3a1d 0 1225_fc42 0
0 2 1 0 2 1 0


450
500
550
600
= G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
0
X
0
x
0
1
x
0
x
0
XXX_XXXX
x
0
0
0 1 0 1 0
0 83 0 58 0
0
0 e959_b00e 0 93e4_ece1 0
0 2 1 0 2 1 0 2


600
650
700
750
= G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
0
X
0
x
0
1
x
0
x
0
XXX_XXXX
x
0
0
0 1 0 2 0 2
71 0 148 0 54
0 94 0
abb0_b331 0 4afd_08d5 0 4a27_2a
0 2 1 0 2 1 0 2 1



APB PROTOCOL WAIT-STATE(N=0)

Simulation Output

UVM_INFO driver.sv(31) @ 5: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=86 write=1 data=3e08d426

UVM_INFO driver.sv(44) @ 15: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=86 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO scoreboard.sv(32) @ 25: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=86 data=3e08d426

UVM_INFO driver.sv(31) @ 35: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=240 write=1 data=c6d1d15e

UVM_INFO scoreboard.sv(32) @ 65: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=240 data=c6d1d15e

UVM_INFO driver.sv(44) @ 65: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=240 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 85: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=73 write=1 data=f1d20d11

UVM_INFO scoreboard.sv(32) @ 115: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=73 data=f1d20d11

UVM_INFO driver.sv(44) @ 115: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=73 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 135: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=191 write=1 data=8dc5a05a

UVM_INFO scoreboard.sv(32) @ 165: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=191 data=8dc5a05a

UVM_INFO driver.sv(44) @ 165: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=191 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 185: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=116 write=1 data=5e8b780d

UVM_INFO scoreboard.sv(32) @ 215: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=116 data=5e8b780d

UVM_INFO driver.sv(44) @ 215: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=116 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 235: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=13 write=0 data=5c213a1d

UVM_INFO scoreboard.sv(36) @ 265: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=13 expected=d actual=d

UVM_INFO driver.sv(44) @ 265: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=13 pwrite=0 prdata=d PSLVERR=0

UVM_INFO driver.sv(31) @ 285: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=211 write=0 data=1225fc42

UVM_INFO scoreboard.sv(36) @ 315: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=211 expected=d3 actual=d3

UVM_INFO driver.sv(44) @ 315: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=211 pwrite=0 prdata=d3 PSLVERR=0

UVM_INFO driver.sv(31) @ 335: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=83 write=0 data=e959b00e

UVM_INFO scoreboard.sv(36) @ 365: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=83 expected=53 actual=53

UVM_INFO driver.sv(44) @ 365: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=83 pwrite=0 prdata=53 PSLVERR=0

UVM_INFO driver.sv(31) @ 385: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=58 write=0 data=93e4ece1

UVM_INFO scoreboard.sv(36) @ 415: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=58 expected=3a actual=3a

UVM_INFO driver.sv(44) @ 415: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=58 pwrite=0 prdata=3a PSLVERR=0

UVM_INFO driver.sv(31) @ 435: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=71 write=0 data=abb0b331

UVM_INFO scoreboard.sv(36) @ 465: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=71 expected=47 actual=47

UVM_INFO driver.sv(44) @ 465: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=71 pwrite=0 prdata=47 PSLVERR=0

UVM_INFO driver.sv(31) @ 485: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=348408063 write=0 data=4afd08d5

UVM_INFO scoreboard.sv(22) @ 505: uvm_test_top.env_o.sb [apb_scoreboard] INVALID ADDRESS PASS: addr=348408063 PSLVERR=1

UVM_INFO driver.sv(44) @ 505: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=348408063 pwrite=0 prdata=0 PSLVERR=1

UVM_INFO test.sv(23) @ 565: uvm_test_top [apb_base_test] END OF TESTCASE

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 565: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 36

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[TEST_DONE] 1

[apb_base_test] 1

[apb_driver] 22

[apb_scoreboard] 11

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

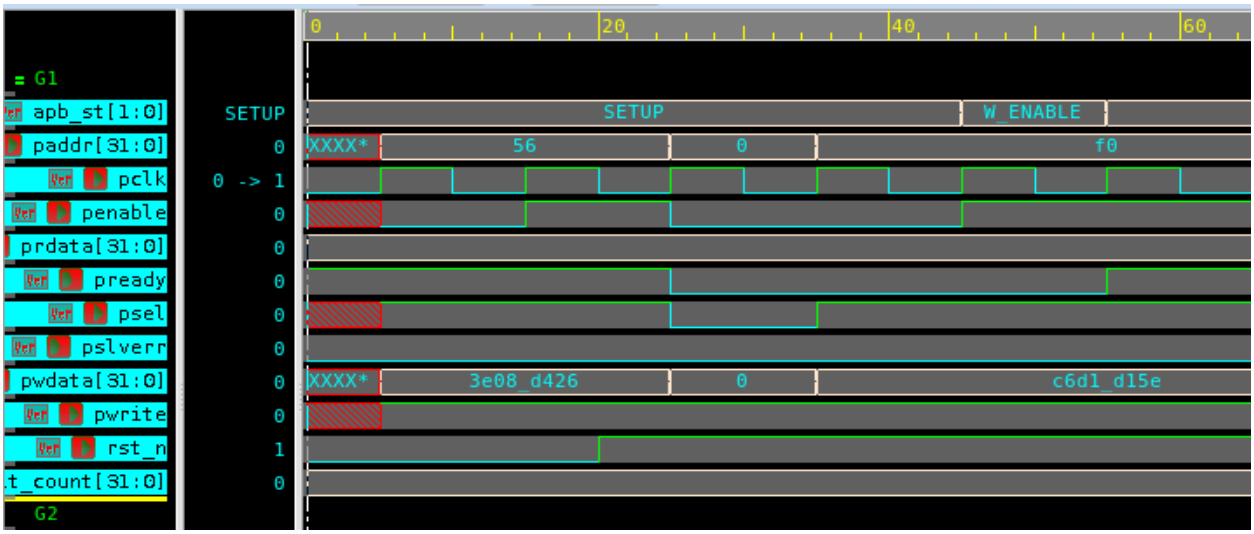
$finish at simulation time 565

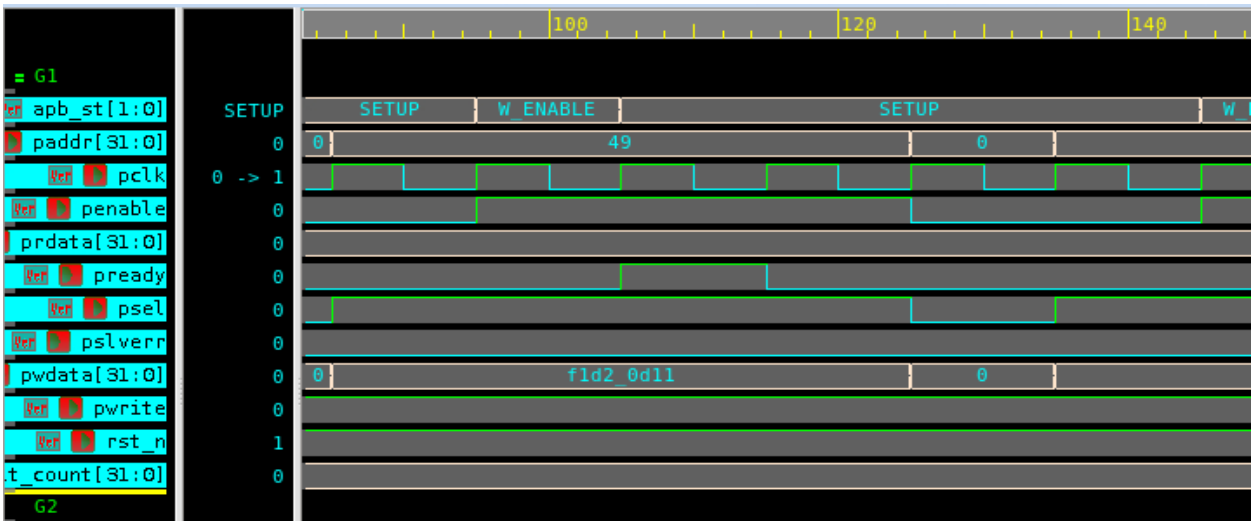
V C S S i m u l a t i o n R e p o r t

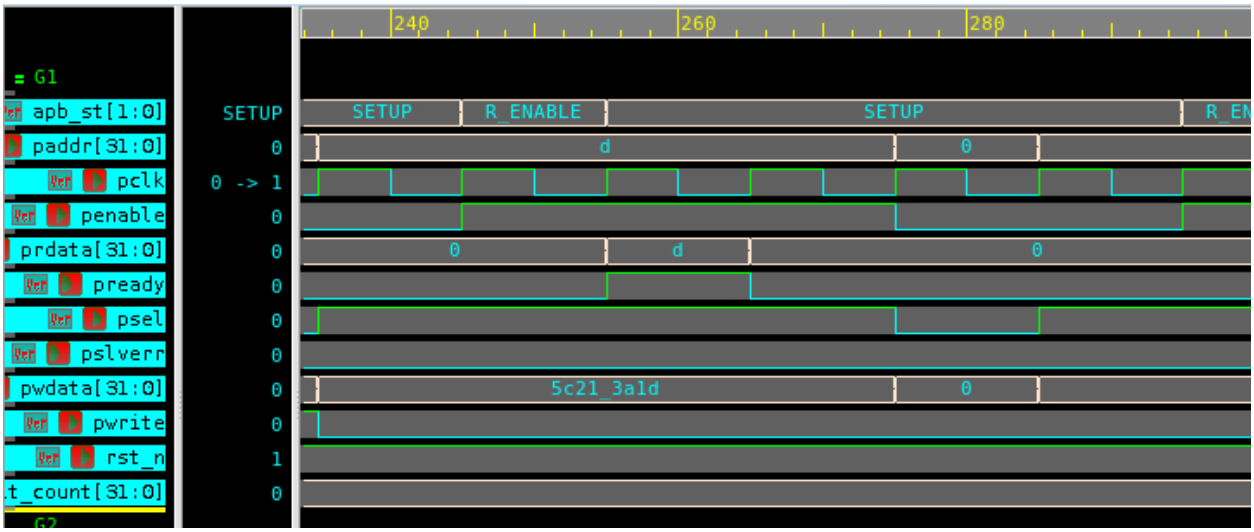
Time: 565

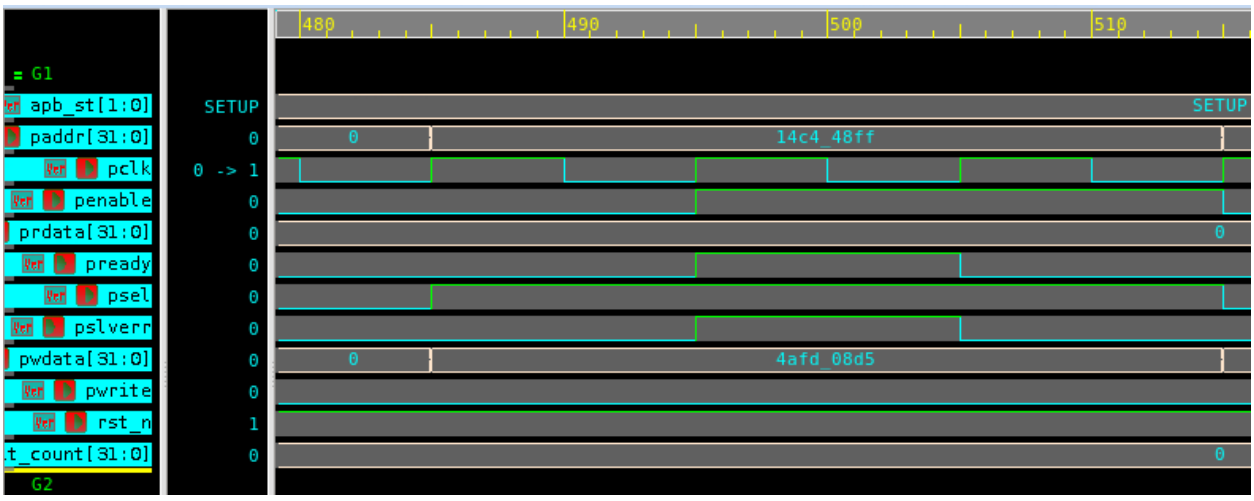
CPU Time: 0.150 seconds; Data structure size: 0.6Mb

Waveform Analysis (wait-state= 0)(min 2 clock cycle)




100
120
140
G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
SETUP
0
0 -> 1
0
0
0
0
0
0
1
0
SETUP
W_ENABLE
SETUP
0
49
0
0
fld2_0d11
0


240
260
280
G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
SETUP
0
0 -> 1
0
0
0
0
0
0
1
0
SETUP
R_ENABLE
SETUP
d
0
0
d
0
5c21_3a1d
0


480
490
500
510
G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
SETUP
0
0 -> 1
0
0
0
0
0
0
1
0
SETUP
0
14c4_48ff
0
0
4afd_08d5
0